

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Dipartimento di Matematica e Geoscienze

Laurea Triennale in Intelligenza Artificiale e Data Analytics

---

TESI DI LAUREA

Applicazione di tecniche di Machine Learning e Deep Learning per la previsione della direzione dei prezzi azionari

Relatore:
Prof. Alejandro Garcia Rodriguez

Laureando:
Riccardo Striano

Anno Accademico 2025/2026

Abstract

Negli ultimi anni l'utilizzo di tecniche di Machine Learning e Deep Learning nell'ambito dei mercati finanziari ha suscitato un crescente interesse grazie alla disponibilità di grandi quantità di dati storici e alla possibilità di individuare pattern utili per la previsione dell'andamento dei prezzi azionari.

L'obiettivo di questo lavoro è analizzare e confrontare diversi modelli di classificazione supervisionata per la previsione della direzione futura del prezzo di un titolo azionario. A tal fine è stato utilizzato un dataset contenente i dati storici del titolo Apple Inc. (AAPL), ottenuti tramite la libreria yfinance e successivamente elaborati attraverso tecniche di preprocessing e feature engineering. In particolare, sono stati calcolati alcuni indicatori tecnici, tra cui medie mobili, volatilità e momentum, utilizzati come variabili di input per i modelli.

Sono stati implementati e confrontati quattro algoritmi: Regressione Logistica, Support Vector Machine, Random Forest e una rete neurale ricorrente di tipo Long Short-Term Memory (LSTM). Le prestazioni dei modelli sono state valutate mediante diverse metriche di classificazione, tra cui accuracy, precision, recall e F1-score, oltre all'analisi delle confusion matrix e dell'importanza delle feature.

I risultati ottenuti evidenziano come la previsione della direzione dei prezzi azionari rappresenti un problema particolarmente complesso. Sebbene alcuni modelli abbiano mostrato prestazioni leggermente superiori rispetto ad altri, nessuno di essi è stato in grado di raggiungere livelli di accuratezza particolarmente elevati. Tali risultati risultano coerenti con la natura fortemente dinamica dei mercati finanziari e con la presenza di numerosi fattori esterni difficilmente modellabili attraverso i soli dati storici.

Il lavoro conferma come le tecniche di Machine Learning e Deep Learning costituiscano strumenti efficaci per l'analisi delle serie temporali finanziarie, pur evidenziando i limiti legati alla previsione dei mercati e l'importanza di un'attenta selezione delle feature e delle metodologie di valutazione.

Indice

Abstract **1**

1 Introduzione **5**
1.1 Contesto e motivazioni 5
1.2 Obiettivo del lavoro 5
1.3 Struttura della tesi 6

2 Fondamenti Teorici **7**
2.1 Machine Learning e problemi di classificazione 7
2.2 Analisi tecnica dei mercati finanziari 8
2.3 Regressione logistica 9
2.3.1 Modello e funzione sigmoide 9
2.4 Support Vector Machine (SVM) 10
2.4.1 Iperpiano di separazione e margine 11
2.4.2 Kernel Trick 12
2.5 Random Forest 12
2.5.1 Alberi decisionali 13
2.5.2 Ensemble learning e majority vote 13
2.6 Reti Neurali Ricorrenti (RNN) 14
2.6.1 Struttura e funzionamento 14
2.6.2 Applicazioni alle serie temporali 15
2.6.3 LSTM e gestione delle dipendenze temporali 15

3 Dataset e preparazione dei dati **16**
3.1 Descrizione del dataset e variabili originali 16
3.2 Esplorazione e statistiche descrittive 16
3.3 Feature engineering 17
3.3.1 Rendimento giornaliero 17
3.3.2 Medie Mobili 18
3.3.3 Volatilità 18
3.3.4 Momentum 18

3.4 Definizione della variabile target 18
3.5 Gestione dei dati mancanti e suddivisione train/test temporale 19

4 Metodologia **20**
4.1 Pipeline sperimentale 20
4.2 Baseline di confronto 20
4.3 Validazione mediante TimeSeriesSplit 20
4.4 Implementazione dei modelli 21
4.4.1 Logistic Regression 21
4.4.2 Support Vector Machine 22
4.4.3 Random Forest 22
4.5 Rete Neurale Ricorrente (LSTM) 22

5 Risultati Sperimentali **24**
5.1 Metriche di valutazione 24
5.2 Confronto tra modelli 24
5.3 Analisi tramite Confusion Matrix 26
5.4 Analisi grafica delle predizioni 27
5.4.1 Prezzo reale e segnali del modello 27
5.4.2 Confronto tra valori reali e predetti 27
5.5 Importanza delle feature 29
5.6 Analisi dell'impatto del feature engineering 30

6 Discussione dei risultati **32**
6.1 Interpretazioni e confronto delle performance 32
6.2 Vantaggi e limiti dei modelli utilizzati 33
6.3 Limiti della previsione nei mercati finanziari 33

7 Conclusioni **35**
7.1 Sintesi del lavoro 35
7.2 Sviluppi futuri 36

Elenco delle figure

2.1 Funzione sigmoide della regressione logistica 10
2.2 Punti di un piano cartesiano definiti come support vector 11
2.3 Confronto tra possibili iperpiani di separazione 11
2.4 Schema generale del funzionamento di una Random Forest 13

3.1 Andamento del prezzo di chiusura del titolo Apple nel periodo analizzato . 17

5.1 Confronto dell'accuracy dei modelli sul test set 25
5.2 Confusion matrix dei modelli sul test set 26
5.3 Andamento del prezzo di Apple e segnali di rialzo predetti dalla Random Forest 28
5.4 Confronto tra la direzione reale e quella predetta dalla Random Forest . . . 28
5.5 Importanza delle feature nella Random Forest 29
5.6 Confronto dell'accuracy dei modelli con e senza feature engineering 31

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

Negli ultimi anni i mercati finanziari hanno generato quantità crescenti di dati disponibili in tempo reale, favorendo l'utilizzo di tecniche computazionali avanzate per l'analisi e la previsione dell'andamento dei prezzi azionari. La previsione dei movimenti di mercato rappresenta un problema particolarmente complesso, poiché i prezzi delle azioni sono influenzati da numerosi fattori economici, politici e psicologici difficilmente modellabili con metodi tradizionali. L'ipotesi dei mercati efficienti, formulata da Fama nel 1970, sostiene che i prezzi riflettono in ogni momento tutte le informazioni disponibili, rendendo teoricamente impossibile ottenere un vantaggio sistematico dall'analisi dei dati storici [7]. Tuttavia, numerosi studi empirici hanno evidenziato la presenza di inefficienze e pattern ricorrenti nei mercati, aprendo la strada all'utilizzo di modelli quantitativi per la previsione dei prezzi. In questo contesto, le tecniche di Machine Learning e Deep Learning si sono affermate come strumenti efficaci per individuare relazioni complesse nei dati storici [8], trovando applicazione crescente nella previsione della direzione futura dei prezzi azionari [9].

1.2 Obiettivo del lavoro

L'obiettivo di questo lavoro è applicare e confrontare diverse tecniche di Machine Learning e Deep Learning per la previsione della direzione dei prezzi azionari utilizzando dati storici di mercato. In particolare, verranno implementati e analizzati diversi modelli di classificazione, tra cui Regressione Logistica, Support Vector Machine, Random Forest e reti neurali ricorrenti di tipo LSTM. I modelli saranno addestrati utilizzando feature ottenute tramite tecniche di feature engineering applicate alle serie storiche finanziarie, come rendimenti giornalieri, medie mobili, volatilità e indicatori di momentum. Successivamente, le performance dei modelli verranno valutate attraverso metriche di classificazione quali

accuracy, precision, recall e F1-score, oltre all'analisi delle confusion matrix e alla rappresentazione grafica delle predizioni ottenute. Lo scopo finale è verificare l'efficacia dei diversi approcci nel riconoscimento dei pattern presenti nei dati finanziari e confrontarne punti di forza e limiti nel contesto della previsione dei mercati azionari.

1.3 Struttura della tesi

Dopo il presente capitolo introduttivo, nel quale sono stati delineati il contesto, le motivazioni e gli obiettivi del lavoro, la tesi è organizzata in ulteriori sei capitoli. Ciascuno di essi approfondisce una fase specifica dello studio, partendo dai fondamenti teorici necessari per la comprensione dei modelli, per poi passare alla descrizione dei dati, alla metodologia sperimentale e all'analisi dei risultati ottenuti. Nel secondo capitolo vengono presentati i principali concetti teorici relativi al Machine Learning, ai problemi di classificazione e alle tecniche utilizzate nel lavoro, con particolare attenzione alla Regressione logistica, alla Support Vector Machine, alla Random Forest e alle reti neurali ricorrenti di tipo LSTM.

Capitolo 2

Fondamenti Teorici

2.1 Machine Learning e problemi di classificazione

Il Machine Learning è un ramo dell'Intelligenza Artificiale che studia algoritmi e modelli in grado di apprendere automaticamente dai dati senza essere esplicitamente programmati. L'obiettivo principale è individuare pattern e relazioni presenti nei dati, così da poter effettuare previsioni o prendere decisioni su nuovi dati non ancora osservati.

Tra le principali categorie di Machine Learning si distinguono:

- Apprendimento supervisionato (supervised learning)
- Apprendimento non supervisionato (unsupervised learning)
- Apprendimento per rinforzo (reinforcement learning)

Nel presente lavoro viene utilizzato l'apprendimento supervisionato, in cui il modello viene addestrato attraverso un dataset composto da dati di input e da una variabile target associata. Durante la fase di training, l'algoritmo apprende una funzione in grado di collegare le caratteristiche di input al valore target, con l'obiettivo di generalizzare correttamente anche su dati mai visti prima. Uno dei problemi più comuni nell'ambito dell'apprendimento supervisionato è la classificazione. In un problema di classificazione, il modello deve assegnare ogni osservazione a una determinata categoria o classe. Nel caso di una classificazione binaria, le possibili classi sono due.

Nel contesto dei mercati finanziari, i problemi di classificazione vengono spesso utilizzati per prevedere la direzione futura del prezzo di un'azione. In particolare, il modello può essere addestrato per stabilire se il prezzo tenderà a salire oppure a scendere in un determinato intervallo temporale. In questo caso la variabile target assume valori binari:

- 1 - se il prezzo aumenta
- 0 - se il prezzo diminuisce

Per effettuare tali previsioni vengono utilizzate diverse feature ottenute dai dati storici di mercato, come rendimenti giornalieri, medie mobili, volatilità e indicatori di momentum. I modelli di Machine Learning cercano quindi di individuare le relazioni tra queste variabili e l'andamento futuro del mercato.

Una delle principali difficoltà nei problemi di classificazione finanziaria è rappresentata dall'elevata variabilità dei dati. Le serie temporali finanziarie sono infatti caratterizzate da rumore, non linearità e forte dipendenza temporale, elementi che rendono complessa la costruzione di modelli accurati e capaci di generalizzare correttamente. Per valutare le performance di un classificatore vengono utilizzate diverse metriche, tra cui accuracy, precision, recall e F1-score. Tali metriche consentono di analizzare non solo il numero di predizioni corrette, ma anche la capacità del modello di distinguere correttamente le diverse classi.

2.2 Analisi tecnica dei mercati finanziari

L'analisi tecnica è una metodologia utilizzata per studiare l'andamento dei mercati finanziari attraverso l'osservazione dei dati storici di prezzo e volume. L'obiettivo principale è individuare pattern e tendenze che possano fornire indicazioni sull'evoluzione futura del mercato. A differenza dell'analisi fondamentale, che si basa sullo studio di fattori economici e finanziari delle aziende, l'analisi tecnica considera che le informazioni rilevanti siano già incorporate nel prezzo di mercato. Per questo motivo l'attenzione viene concentrata principalmente sull'andamento temporale dei prezzi e sui movimenti del mercato.

Gli strumenti più utilizzati nell'analisi tecnica sono gli indicatori tecnici, cioè misure matematiche calcolate a partire dai dati storici. Tra gli indicatori maggiormente utilizzati vi sono:

- medie mobili
- volatilità
- momentum
- indicatori di trend

Le medie mobili vengono utilizzate per ridurre il rumore presente nelle serie storiche e individuare la direzione generale del mercato. In particolare, una media mobile rappresenta il valore medio del prezzo in un determinato intervallo temporale e consente di evidenziare eventuali tendenze rialziste o ribassiste. La volatilità misura invece il grado di variazione dei prezzi nel tempo e rappresenta un indicatore del livello di rischio associato a un'attività finanziaria. Valori elevati di volatilità indicano generalmente una maggiore instabilità del mercato. Il momentum è un indicatore utilizzato per valutare la

velocità con cui il prezzo di un'azione varia nel tempo. Questo tipo di misura permette di individuare la forza di un trend e possibili inversioni di direzione.

2.3 Regressione logistica

La regressione logistica è uno degli algoritmi di classificazione binaria più utilizzati nel Machine Learning supervisionato. A differenza della regressione lineare, che produce un valore numerico continuo, la regressione logistica fornisce un risultato interpretabile come probabilità di appartenenza a una specifica classe, rendendola particolarmente adatta ai problemi in cui l'obiettivo è assegnare un'osservazione a una delle due categorie possibili.

Questo modello viene ampiamente utilizzato nei problemi di classificazione supervisionata grazie alla sua semplicità, interpretabilità ed efficienza computazionale. Nel contesto finanziario, la regressione logistica può essere applicata per prevedere la direzione futura del prezzo di un'azione, classificando i movimenti di mercato come rialzisti o ribassisti sulla base delle feature estratte dai dati storici.

2.3.1 Modello e funzione sigmoide

Il cuore della regressione logistica è la funzione sigmoide (o logistica), definita come:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2.1}$$

dove z rappresenta una combinazione lineare delle variabili di input.

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n \tag{2.2}$$

Come illustrato in Figura 2.1, la funzione sigmoide trasforma qualsiasi valore reale in un valore compreso tra 0 e 1, rendendo il risultato interpretabile come una probabilità. In particolare:

- Valori vicini a 1 indicano alta probabilità della classe positiva
- Valori vicini a 0 indicano alta probabilità della classe negativa

La decisione finale viene generalmente presa applicando una soglia (tipicamente 0.5) per cui:

- Se $\sigma(z) \geq 0.5$, allora la classe predetta è 1.
- Se $\sigma(z) < 0.5$, allora la classe predetta è 0.

Grazie alla sua semplicità e alla facilità di interpretazione dei risultati, la regressione logistica viene spesso impiegata come modello di riferimento nei problemi di classificazione, rappresentando un termine di confronto utile per valutare le performance di modelli più complessi [1].

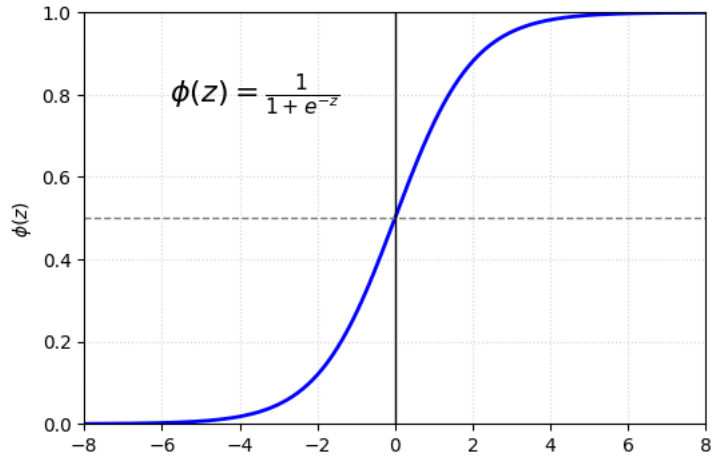


Figura 2.1: Funzione sigmoide della regressione logistica

2.4 Support Vector Machine (SVM)

La Support Vector Machine (SVM) è un algoritmo di apprendimento supervisionato utilizzato principalmente per problemi di classificazione. Grazie alla sua efficacia e versatilità, trova applicazione in numerosi ambiti, tra cui l'elaborazione del linguaggio naturale, il riconoscimento vocale e delle immagini e la computer vision.

Il principio fondamentale della SVM consiste nell'individuare un iperpiano che separi al meglio le osservazioni appartenenti a classi diverse [2]. In questo contesto assumono un ruolo centrale i vettori di supporto, ovvero le osservazioni più vicine all'iperpiano di separazione. Tali punti sono determinanti per la definizione del confine decisionale: una loro modifica comporta una variazione nella posizione dell'iperpiano.

Un concetto chiave nella SVM è il margine, definito come la distanza tra i vettori di supporto delle due classi e l'iperpiano. L'obiettivo dell'algoritmo è massimizzare questo margine, poiché un margine più ampio è associato a una maggiore capacità di generalizzazione del modello. La Support Vector Machine ha l'obiettivo di identificare l'iperpiano che meglio divide i vettori di supporto in classi. Per farlo segue i seguenti passaggi:

1. Ricerca di un iperpiano linearmente separabile che divida correttamente le osservazioni delle due classi. Se esistono più iperpiani validi, viene selezionato quello con il margine massimo rispetto ai vettori di supporto.

2. Se tale iperpiano non esiste, la SVM utilizza una mappatura non lineare per trasformare i dati in uno spazio a dimensionalità superiore, in cui la separazione lineare diventa possibile

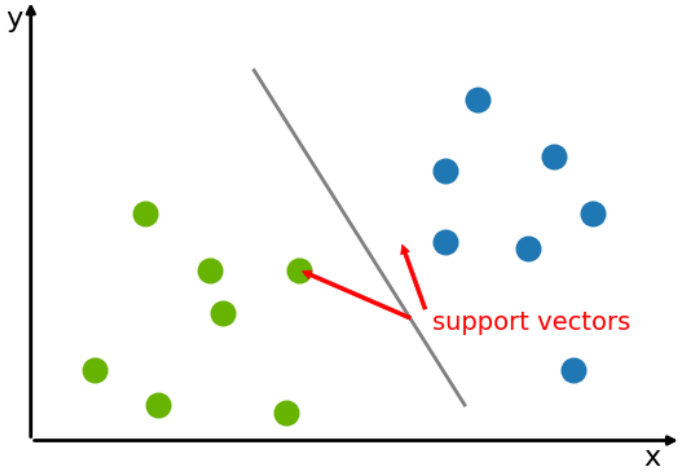


Figura 2.2: Punti di un piano cartesiano definiti come support vector

2.4.1 Iperpiano di separazione e margine

Nel caso dell'iperpiano linearmente separabile è possibile avere, teoricamente, un numero infinito di rette per separare le osservazioni. Il problema è trovare quale tra le infinite rette risulti ottimale, ossia quella che generi il minimo errore di classificazione su una nuova osservazione. Infatti, più lontano dall'iperpiano si trovano i nostri punti dati, più siamo fiduciosi che essi siano stati classificati correttamente. Pertanto, desideriamo che i nostri punti osservati siano il più lontano possibile dall'iperpiano, pur rimanendo nella parte corretta. Quindi, quando vengono aggiunti nuovi dati di test, il modello decide la classe da assegnare a ciascuna osservazione. Questo è ciò che fa il Support Vector Machine nella pratica, quando trova un iperpiano linearmente separabile [3].

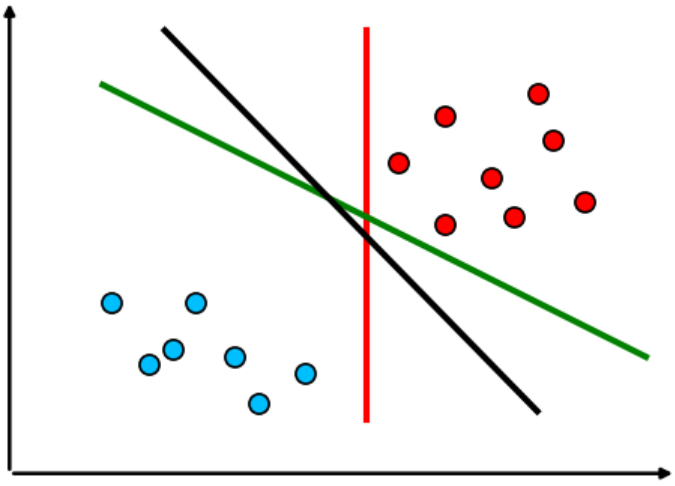


Figura 2.3: Confronto tra possibili iperpiani di separazione

L'equazione generale dell'iperpiano può essere espressa come:

$$w \cdot x + b = 0 \tag{2.3}$$

Dove:

- w rappresenta il vettore dei pesi
- x il vettore delle feature
- b il termine bias

2.4.2 Kernel Trick

In molti problemi reali le osservazioni non risultano linearmente separabili nello spazio originale delle feature. In questi casi la SVM utilizza il cosiddetto kernel trick, una tecnica che consente di trasformare implicitamente i dati in uno spazio a dimensionalità più elevata in cui la separazione lineare diventa possibile. L'idea principale consiste nell'applicare una trasformazione alle osservazioni originali tramite una funzione $\phi(x)$, che proietta i dati in un nuovo spazio delle caratteristiche (feature space). In questo nuovo spazio, le classi possono risultare separabili tramite un iperpiano lineare anche se nello spazio originale presentano una distribuzione non lineare.

Formalmente, invece di calcolare esplicitamente la trasformazione dei dati, la SVM utilizza una funzione kernel $K(x_i, x_j)$ che misura la similarità tra due osservazioni [1]:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j) \tag{2.4}$$

Questo approccio permette di effettuare i calcoli direttamente nello spazio originale senza dover determinare esplicitamente le coordinate nello spazio trasformato, riducendo notevolmente lo spazio computazionale.

2.5 Random Forest

La Random Forest è un algoritmo di Machine Learning supervisionato appartenente alla categoria dei metodi ensemble. L'idea principale consiste nel combinare le predizioni di più classificatori per ottenere risultati generalmente più accurati e stabili rispetto a quelli prodotti da un singolo modello. Nel caso della Random Forest, il modello è costituito da un insieme di alberi decisionali costruiti indipendentemente tra loro. Ogni albero viene addestrato su una porzione casuale del dataset e utilizza un sottoinsieme casuale delle feature disponibili [4]. Questo approccio introduce diversità tra gli alberi e contribuisce a migliorare la capacità di generalizzazione del modello. Nel problema di classificazione, la

previsione finale viene ottenuta tramite un meccanismo di majority voting: ogni albero produce una predizione e la classe più votata rappresenta il risultato finale del modello. Uno dei principali vantaggi della Random Forest è la capacità di ridurre il rischio di overfitting rispetto a un singolo albero decisionale. Mentre un singolo albero può adattarsi eccessivamente ai dati di addestramento, la combinazione di numerosi alberi consente di ottenere predizioni più robuste e meno sensibili al rumore presente nei dati [5].

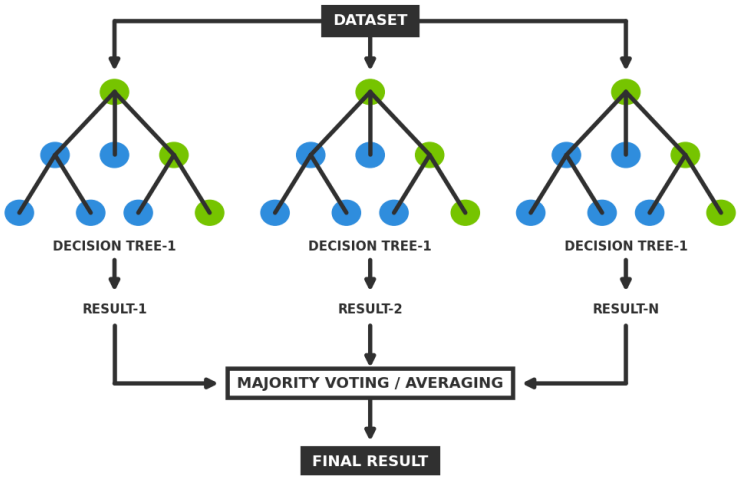


Figura 2.4: Schema generale del funzionamento di una Random Forest

2.5.1 Alberi decisionali

Un albero decisionale (Decision Tree) è un modello predittivo che suddivide progressivamente i dati attraverso una sequenza di decisioni. Ogni nodo dell'albero rappresenta una condizione applicata a una determinata feature, mentre i rami rappresentano i possibili risultati della decisione. Il nodo iniziale viene chiamato nodo radice (root node), mentre i nodi intermedi sono definiti nodi decisionali (decision nodes). Il processo termina nei nodi foglia (leaf nodes), che rappresentano la classificazione finale. Durante l'addestramento, l'albero cerca di individuare le suddivisioni che permettono di separare nel modo più efficace le diverse classi del dataset. Nonostante la semplicità interpretativa, i singoli alberi decisionali tendono facilmente all'overfitting, motivo per cui vengono spesso utilizzati all'interno dei modelli ensemble come la Random Forest.

2.5.2 Ensemble learning e majority vote

La Random Forest appartiene ai modelli di ensemble learning, tecniche che combinano più modelli predittivi per migliorare le prestazioni complessive del sistema. Nel caso della classificazione, ogni Decision Tree produce una previsione indipendente e la classe finale viene determinata tramite majority vote, ovvero selezionando la classe maggiormente predetta dagli alberi della foresta.

Per costruire una Random Forest è necessario definire alcuni parametri fondamentali [6]:

- Numero di alberi `n_estimators`: indica quanti alberi compongono la foresta. Un numero maggiore di alberi può migliorare l'accuratezza del modello, aumentando però il tempo di calcolo;
- Profondità massima `max_depth`: definisce la profondità massima consentita per ciascun albero. Valori elevati permettono di apprendere relazioni più complesse, ma possono aumentare il rischio di overfitting;
- Peso delle classi `class_weight`: consente di assegnare pesi differenti alle classi in presenza di dataset sbilanciati, migliorando la gestione delle classi meno rappresentate.

Nel presente lavoro, la Random Forest è stata utilizzata per classificare la direzione futura dei prezzi azionari sulla base delle feature estratte dai dati storici di mercato. Grazie alla capacità di modellare relazioni non lineari e interazioni complesse tra le variabili, questo algoritmo rappresenta uno dei modelli più efficaci nell'ambito della classificazione finanziaria.

2.6 Reti Neurali Ricorrenti (RNN)

Le reti neurali ricorrenti (Recurrent Neural Networks, RNN) sono una particolare tipologia di reti neurali progettata per elaborare dati sequenziali e serie temporali. A differenza delle reti neurali tradizionali, le RNN sono in grado di mantenere informazioni provenienti dagli input precedenti, rendendole particolarmente adatte all'analisi di dati dipendenti dal tempo. Questo tipo di architettura trova applicazione in numerosi ambiti, come l'elaborazione del linguaggio naturale, il riconoscimento vocale e la previsione di serie temporali finanziarie.

2.6.1 Struttura e funzionamento

Nelle reti neurali ricorrenti, l'output prodotto in un determinato istante dipende non solo dall'input corrente, ma anche dalle informazioni elaborate negli istanti precedenti. Questo comportamento è reso possibile dalla presenza di connessioni ricorrenti che permettono alla rete di mantenere una sorta di memoria interna. A ogni passo temporale, la rete aggiorna il proprio stato nascosto combinando il nuovo input con le informazioni memorizzate in precedenza. In questo modo il modello può apprendere dipendenze temporali presenti nei dati sequenziali.

2.6.2 Applicazioni alle serie temporali

Le RNN vengono frequentemente utilizzate nell'analisi delle serie temporali grazie alla loro capacità di modellare dati ordinati nel tempo. Nel contesto finanziario, tali reti possono essere impiegate per analizzare l'andamento storico dei prezzi azionari e individuare possibili pattern temporali utili alla previsione dei movimenti futuri del mercato. A differenza di molti algoritmi tradizionali di Machine Learning, le RNN riescono a considerare direttamente la dipendenza temporale tra le osservazioni consecutive, caratteristica particolarmente importante nei dati finanziari. Tuttavia, le RNN standard presentano alcune limitazioni quando è necessario apprendere relazioni temporali di lungo periodo. Durante l'addestramento può verificarsi il problema del vanishing gradient, che riduce la capacità della rete di mantenere informazioni provenienti da input molto distanti nel tempo.

2.6.3 LSTM e gestione delle dipendenze temporali

Per superare i limiti delle RNN tradizionali sono state introdotte le reti LSTM (Long Short-Term Memory), un'architettura progettata per gestire in modo più efficace le dipendenze temporali a lungo termine [8]. A differenza delle RNN standard, le LSTM introducono una cella di memoria (cell state), che scorre lungo l'intera sequenza temporale e ha il compito di conservare le informazioni rilevanti apprese nei passi precedenti. Il contenuto di questa cella viene aggiornato in modo selettivo attraverso tre meccanismi di controllo, chiamati gate, ciascuno dei quali decide quanta informazione lasciar passare. Il forget gate stabilisce quali informazioni presenti nella cella di memoria devono essere mantenute e quali invece devono essere scartate, poiché non più rilevanti per la previsione corrente. Questo meccanismo è fondamentale per evitare che la rete accumuli informazioni obsolete o ridondanti lungo la sequenza. L'input gate regola invece quanta nuova informazione, proveniente dall'osservazione corrente, deve essere incorporata nella cella di memoria. In questo modo la rete è in grado di aggiornare la propria conoscenza interna integrando i dati più recenti con quelli accumulati in precedenza. Infine, l'output gate determina quale parte del contenuto della cella di memoria deve essere effettivamente utilizzata per produrre lo stato nascosto, ovvero l'output che la rete trasmette al passo temporale successivo e che, nell'ultimo istante della sequenza, viene utilizzato per effettuare la previsione finale.

Grazie a questa struttura, le LSTM riescono a mantenere e propagare informazioni rilevanti anche attraverso sequenze temporali molto lunghe, senza che queste vengano progressivamente perse, come invece accade nelle RNN tradizionali a causa del problema del vanishing gradient. Questa caratteristica rende le LSTM particolarmente adatte all'analisi delle serie storiche finanziarie, nelle quali le informazioni passate possono influenzare l'andamento futuro del mercato anche a distanza di tempo [8].

Capitolo 3

Dataset e preparazione dei dati

3.1 Descrizione del dataset e variabili originali

Per lo sviluppo del presente lavoro è stato utilizzato un dataset contenente dati storici relativi al titolo azionario Apple Inc. (AAPL). I dati sono stati ottenuti tramite la libreria Python *yfinance*, che consente di scaricare informazioni finanziarie storiche da Yahoo Finance [10]. Il dataset copre il periodo compreso tra il 2 gennaio 2018 e il 29 dicembre 2023 ed è organizzato come una serie temporale giornaliera. Ogni osservazione rappresenta una giornata di contrattazione del titolo azionario.

Le variabili originali presenti nel dataset sono:

- Open: prezzo di apertura
- High: prezzo massimo giornaliero
- Low: prezzo minimo giornaliero
- Close: prezzo di chiusura
- Volume: volume degli scambi

Tra queste variabili, il prezzo di chiusura assume particolare importanza poiché viene utilizzato per il calcolo di diversi indicatori tecnici e della variabile target utilizzata nei modelli di classificazione. I dati raccolti sono stati successivamente elaborati attraverso tecniche di preprocessing e feature engineering, così da renderli adatti all'applicazione degli algoritmi di Machine Learning e Deep Learning analizzati nel presente lavoro.

3.2 Esplorazione e statistiche descrittive

Prima della fase di addestramento dei modelli è stata effettuata un'analisi esplorativa del dataset, con l'obiettivo di comprendere meglio le caratteristiche dei dati e verificarne la qualità. In particolare, sono state analizzate:

- la dimensione del dataset
- la tipologia delle variabili
- la presenza di eventuali valori mancanti
- alcune statistiche descrittive, come media, deviazione standard, valore minimo e massimo.

Successivamente è stato rappresentato graficamente l'andamento del prezzo di chiusura del titolo Apple nel tempo, così da evidenziare trend, variazioni improvvise e periodi di maggiore volatilità. L'analisi esplorativa ha permesso di osservare il comportamento generale della serie storica e di individuare le informazioni più rilevanti per la fase successiva di feature engineering.



Figura 3.1: Andamento del prezzo di chiusura del titolo Apple nel periodo analizzato

3.3 Feature engineering

Per migliorare la capacità predittiva dei modelli sono state create nuove variabili derivate dai dati originali attraverso tecniche di feature engineering [9]. L'obiettivo di questa fase è fornire ai modelli informazioni aggiuntive utili a identificare pattern presenti nella serie temporale finanziaria.

3.3.1 Rendimento giornaliero

Il rendimento giornaliero misura la variazione percentuale del prezzo di chiusura rispetto al giorno precedente ed è stato utilizzato per analizzare le variazioni quotidiane del titolo azionario. Il rendimento può essere espresso come:

$$Return_t = \frac{Close_t - Close_{t-1}}{Close_{t-1}} \tag{3.1}$$

Questa variabile consente di rappresentare in modo più efficace le variazioni relative del prezzo nel tempo.

3.3.2 Medie Mobili

Sono state calcolate due medie mobili del prezzo di chiusura:

- media mobile a 10 giorni (MA10)
- media mobile a 50 giorni (MA50)

Le medie mobili vengono utilizzate nell'analisi tecnica per individuare il trend generale del mercato e ridurre il rumore presente nelle serie storiche finanziarie [11].

3.3.3 Volatilità

La volatilità è stata calcolata come deviazione standard dei rendimenti in una finestra temporale mobile di 10 giorni. Questa misura permette di quantificare il livello di variabilità del mercato nel tempo. Valori elevati di volatilità indicano generalmente una maggiore instabilità del prezzo azionario, mentre valori bassi segnalano un mercato più stabile [11].

3.3.4 Momentum

Il momentum rappresenta la velocità di variazione del prezzo nel tempo ed è stato calcolato come differenza tra il prezzo corrente e quello registrato in un intervallo temporale precedente. Questo indicatore viene frequentemente utilizzato nell'analisi tecnica per identificare la forza di un trend rialzista o ribassista [11].

3.4 Definizione della variabile target

L'obiettivo del lavoro consiste nella previsione della direzione futura del prezzo azionario attraverso un problema di classificazione binaria [9]. La variabile target è stata definita confrontando il prezzo di chiusura di un giorno con quello del giorno successivo:

- valore 1 se il prezzo del giorno successivo risulta maggiore del prezzo corrente;
- valore 0 se il prezzo del giorno successivo risulta minore o uguale.

In questo modo i modelli vengono addestrati per prevedere se il mercato tenderà a salire oppure a scendere nella giornata successiva.

3.5 Gestione dei dati mancanti e suddivisione train/test temporale

Durante la fase di preprocessing sono stati rimossi i valori mancanti generati dal calcolo degli indicatori tecnici e delle medie mobili. Questo passaggio è necessario poiché gli indicatori calcolati su finestre temporali mobili producono NaN nelle prime osservazioni del dataset.

Successivamente il dataset è stato suddiviso in training set e test set mantenendo l'ordine temporale delle osservazioni, così da evitare fenomeni di data leakage [8]. In particolare, l'80% dei dati è stato utilizzato per l'addestramento dei modelli, mentre il restante 20% è stato utilizzato per la fase di test.

Capitolo 4

Metodologia

4.1 Pipeline sperimentale

La metodologia adottata nel presente lavoro segue una pipeline strutturata in fasi sequenziali. A partire dai dati storici del titolo azionario Apple Inc., sono state applicate tecniche di preprocessing e feature engineering per costruire il dataset di input. Successivamente sono stati addestrati e confrontati quattro modelli di classificazione: Regressione Logistica, Support Vector Machine, Random Forest e una rete neurale ricorrente di tipo LSTM. Le performance dei modelli sono state valutate sul test set attraverso metriche standard di classificazione.

4.2 Baseline di confronto

Prima di addestrare i modelli è stata definita una baseline di riferimento, che rappresenta il termine di confronto minimo per valutare l'effettivo contributo informativo dei modelli di Machine Learning. La baseline adottata consiste in un classificatore banale che prevede sempre la classe 1, ovvero un rialzo del prezzo, indipendentemente dalle feature di input. Questa strategia simula il comportamento di un investitore che assume sempre una visione ottimistica sul mercato.

La baseline ha ottenuto un'accuracy del 53.8% sul test set, corrispondente alla percentuale di giornate di rialzo presenti nel periodo di test. Qualsiasi modello con un reale valore predittivo dovrebbe superare questo valore.

4.3 Validazione mediante TimeSeriesSplit

Prima dell'addestramento dei modelli è stata effettuata una fase di validazione mediante TimeSeriesSplit, una tecnica di cross validation specificamente progettata per l'analisi di serie temporali. A differenza della tradizionale K-Fold Cross Validation, che suddivide ca-

sualmente il dataset in più sottoinsiemi, la TimeSeriesSplit mantiene l'ordine cronologico delle osservazioni. Questa caratteristica è fondamentale nell'ambito della previsione dei mercati finanziari, poiché consente di evitare fenomeni di data leakage, ovvero l'utilizzo di informazioni future durante la fase di addestramento del modello. La procedura consiste nel suddividere progressivamente il training set in più fold temporali. A ogni iterazione il modello viene addestrato utilizzando esclusivamente osservazioni appartenenti al passato e validato su dati temporalmente successivi. In questo modo viene simulato uno scenario realistico di previsione, nel quale il modello dispone solamente delle informazioni disponibili fino a uno specifico istante temporale. Nel presente lavoro è stata utilizzata una TimeSeriesSplit con cinque fold, applicata ai modelli di Machine Learning implementati (Regressione Logistica, Support Vector Machine e Random Forest). Per ciascun modello è stata calcolata l'accuracy ottenuta in ogni fold, insieme alla media e alla deviazione standard delle prestazioni. Questa procedura ha consentito di valutare la stabilità dei modelli durante l'addestramento. Come emerge dai risultati, le prestazioni sono relativamente stabili tra i diversi fold, ma rimangono complessivamente modeste, confermando la difficoltà di prevedere la direzione giornaliera del prezzo azionario.

Modello	**Accuracy media**	**Deviazione standard**
Logistic Regression	48.25%	3.12%
SVM	48.14%	2.82%
Random Forest	48.14%	3.55%

Tabella 4.1: Risultati della validazione temporale mediante TimeSeriesSplit a cinque fold

4.4 Implementazione dei modelli

Tutti i modelli di classificazione classici sono stati implementati tramite la libreria scikit-learn [6], mentre la rete LSTM è stata sviluppata utilizzando il framework Keras incluso in TensorFlow.

4.4.1 Logistic Regression

La Regressione Logistica è stata implementata mediante una pipeline composta da una fase di standardizzazione delle feature, effettuata tramite StandardScaler, e dal classificatore vero e proprio, configurato con un numero massimo di 1000 iterazioni per garantire la convergenza. La standardizzazione è stata adottata poiché le variabili di input presentano scale differenti, in particolare il volume degli scambi. Per evitare fenomeni di data leakage, lo scaler è stato addestrato esclusivamente sul training set e successivamente applicato al test set.

4.4.2 Support Vector Machine

Prima dell'addestramento della SVM, le feature sono state standardizzate mediante StandardScaler, poiché il modello è particolarmente sensibile alla scala delle variabili di input. I parametri dello scaler sono stati calcolati esclusivamente sul training set e successivamente applicati al test set, evitando così l'utilizzo di informazioni provenienti da dati destinati alla valutazione finale.
La configurazione del modello è stata selezionata mediante una ricerca a griglia effettuata con GridSearchCV. La procedura ha confrontato differenti funzioni kernel e diversi valori degli iperparametri C, γ e degree. Per rispettare l'ordine cronologico delle osservazioni è stata effettuata una TimeSeriesSplit con cinque fold, adottando come criterio di selezione l'F1-score ponderato. La configurazione migliore è risultata composta da un kernel RBF, $C = 10$ e $\gamma = 0.1$. Il modello così configurato è stato quindi addestrato sull'intero training set e valutato sul test set.

4.4.3 Random Forest

Anche per la Random Forest è stata effettuata un'ottimizzazione degli iperparametri mediante GridSearchCV. La ricerca ha considerato differenti valori per il numero di alberi, la profondità massima, il numero di osservazioni richieste per suddividere un nodo, il numero minimo di osservazioni presenti nelle foglie e il peso assegnato alle classi. La validazione è stata condotta mediante una TimeSeriesSplit con cinque fold, utilizzando l'F1-score ponderato come criterio di selezione.
La configurazione migliore è risultata composta da 100 alberi decisionali, profondità massima pari a 10, almeno 5 osservazioni necessarie per suddividere un nodo, almeno 2 osservazioni per ciascuna foglia e pesi bilanciati tra le classi. È stato inoltre impostato un seed casuale pari a 42, così da garantire la riproducibilità dei risultati. Il modello selezionato è stato infine addestrato sull'intero training set e valutato sul test set [6].

4.5 Rete Neurale Ricorrente (LSTM)

Per confrontare gli algoritmi tradizionali di Machine Learning con un modello di Deep Learning, è stata implementata una rete neurale ricorrente di tipo Long Short-Term Memory (LSTM). Questa architettura è progettata per elaborare dati sequenziali e mantenere informazioni provenienti dagli istanti temporali precedenti, risultando pertanto adatta all'analisi delle serie storiche finanziarie.
Prima dell'addestramento, le feature sono state standardizzate mediante StandardScaler. I parametri dello scaler, ossia media e deviazione standard, sono stati calcolati esclusivamente sul training set e successivamente applicati al test set, evitando così l'introduzione di informazioni future e possibili fenomeni di data leakage.

Il dataset è stato quindi trasformato in sequenze temporali mediante una finestra mobile di 10 osservazioni. Questa procedura ha prodotto un training set composto da 1.158 sequenze e un test set composto da 282 sequenze, ciascuna di forma (10, 10): la prima dimensione rappresenta il numero di istanti temporali considerati, mentre la seconda corrisponde alle dieci feature utilizzate dal modello.
L'architettura della rete, riportata nella Tabella 4.2, è composta da un livello LSTM con 64 unità, seguito da un livello Dropout con tasso pari a 0.2, introdotto per ridurre il rischio di overfitting. Successivamente è presente un livello Dense con 32 neuroni e una funzione di attivazione ReLU. Il livello di output è costituito da un singolo neurone con funzione di attivazione sigmoide, che restituisce una probabilità compresa tra 0 e 1. Complessivamente, la rete presenta 21.313 parametri addestrabili [12].

Tabella 4.2: Architettura della rete LSTM implementata

Livello	**Configurazione**	**Attivazione**	**Output**	**Parametri**
LSTM	64 unità	tanh/sigmoid	(None, 64)	19.200
Dropout	Tasso 0,2	–	(None, 64)	0
Dense	32 neuroni	ReLU	(None, 32)	2.080
Dense di output	1 neurone	Sigmoid	(None, 1)	33
			Totale parametri addestrabili	**21.313**

Per l'addestramento è stato utilizzato l'ottimizzatore Adam, scelto per la sua capacità di adattare il tasso di apprendimento durante il processo di ottimizzazione. Come funzione di perdita è stata adottata la Binary Crossentropy, comunemente utilizzata nei problemi di classificazione binaria. Il modello è stato addestrato per 20 epoche, utilizzando un batch size pari a 32.
Al termine dell'addestramento, la probabilità prodotta dalla rete è stata convertita in una previsione binaria applicando una soglia pari a 0,5. Le osservazioni con probabilità superiore a tale soglia sono state classificate come classe 1, corrispondente a un rialzo del prezzo, mentre le restanti osservazioni sono state assegnate alla classe 0, corrispondente a un ribasso. Le predizioni ottenute sono state infine confrontate con quelle degli altri modelli mediante le metriche illustrate nel capitolo successivo.

Capitolo 5

Risultati Sperimentali

5.1 Metriche di valutazione

Per valutare le performance dei modelli di classificazione sono state utilizzate diverse metriche comunemente impiegate nei problemi di Machine Learning [9].

In particolare, sono state considerate:

- Accuracy: misura la percentuale di predizioni corrette effettuate dal modello sul totale delle osservazioni
- Precision: indica la capacità del modello di effettuare predizioni positive corrette, ovvero tra tutte le volte che il modello predice un rialzo, quante volte ha ragione
- Recall: misura la capacità di identificare correttamente le osservazioni appartenenti alla classe positiva, ovvero tra tutti i rialzi reali, quanti ne cattura il modello
- F1-score: rappresenta la media armonica tra precision e recall, fornendo una misura bilanciata delle performance del modello

L'utilizzo congiunto di queste metriche consente di ottenere una valutazione più completa delle performance dei modelli, evitando di basarsi esclusivamente sull'accuracy. Questo è particolarmente importante nei problemi di classificazione finanziaria, dove le classi possono essere sbilanciate e l'accuracy da sola può risultare fuorviante [8].

5.2 Confronto tra modelli

I modelli implementati nel presente lavoro sono stati confrontati sulla base delle metriche di classificazione ottenute sul test set. La tabella che segue riporta i risultati ottenuti dai diversi algoritmi analizzati, utilizzando i valori medi ponderati (weighted average) di precision, recall e F1-score.

Dall'analisi dei risultati emergono alcune osservazioni rilevanti:

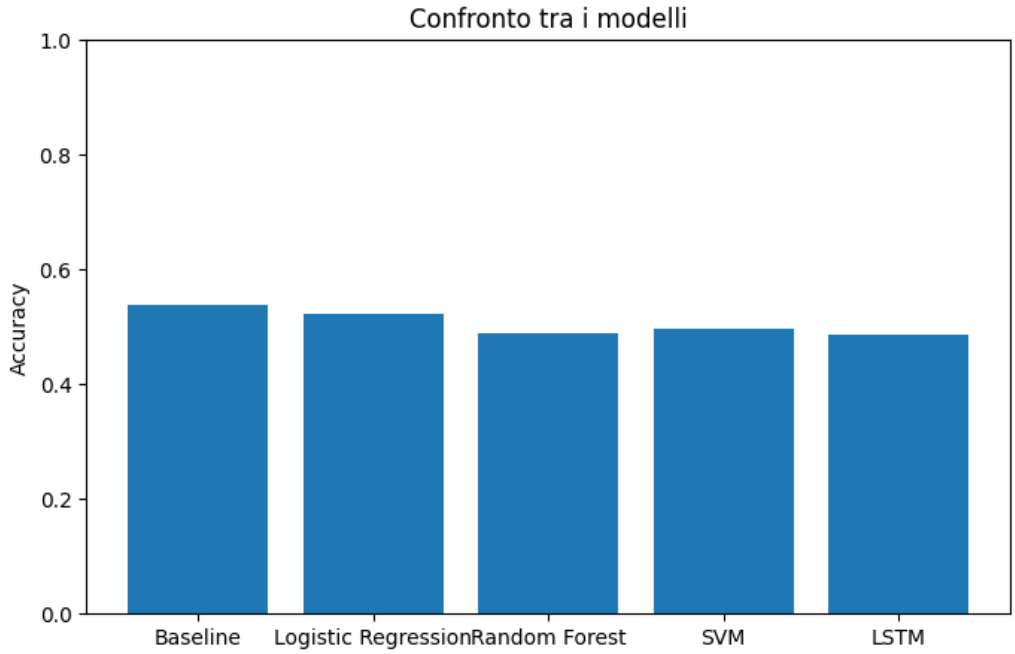


Figura 5.1: Confronto dell'accuracy dei modelli sul test set

Modello	**Accuracy**	**Precision**	**Recall**	**F1-score**
Baseline	53.8%	0.29	0.54	0.38
Logistic Regression	52.4%	0.53	0.52	0.52
SVM	49.7%	0.53	0.50	0.46
Random Forest	49.0%	0.52	0.49	0.45
LSTM	48.6%	0.53	0.49	0.42

Tabella 5.1: Confronto delle performance dei modelli sul test set.

1. La Regressione Logistica ha ottenuto un'accuracy del 52.4%, leggermente inferiore alla baseline del 53,8%. Tuttavia, a differenza della baseline, il modello ha classificato entrambe le classi, ottenendo valori ponderati di precision, recall e F1-score rispettivamente pari a 0.53, 0.52 e 0.52. La standardizzazione delle feature ha quindi determinato un comportamento più bilanciato, pur senza consentire al modello di superare la baseline in termini di accuracy.

2. La Support Vector Machine e la Random Forest hanno ottenuto F1-score ponderati rispettivamente pari a 0.46 e 0.45, superiori al valore di 0.38 registrato dalla baseline, ma inferiori allo 0.52 ottenuto dalla Regressione Logistica. In particolare, la SVM e la Random Forest hanno mostrato una maggiore propensione a identificare le giornate di ribasso, raggiungendo per la classe 0 valori di recall rispettivamente pari a 0.79 e 0.77. Tale risultato è stato tuttavia accompagnato da una minore capacità di riconoscere le giornate di rialzo.

3. La rete LSTM ha ottenuto un'accuracy leggermente inferiore agli altri modelli (48.6%), con un F1-score di 0.42. Nonostante la capacità teorica di catturare dipen-

denze temporali nelle serie storiche, il modello non ha mostrato vantaggi significativi rispetto agli approcci classici nel contesto specifico di questo lavoro.

In generale, tutti i modelli si attestano su valori di accuracy compresi tra il 49% e il 54%, un risultato coerente con quanto riportato per problemi di previsione della direzione dei prezzi azionari [8].

5.3 Analisi tramite Confusion Matrix

Per comprendere meglio il comportamento dei modelli sono state analizzate le confusion matrix ottenute durante la fase di test. La confusion matrix consente di confrontare le classi reali con quelle predette dal modello, distinguendo veri positivi, veri negativi, falsi positivi e falsi negativi.

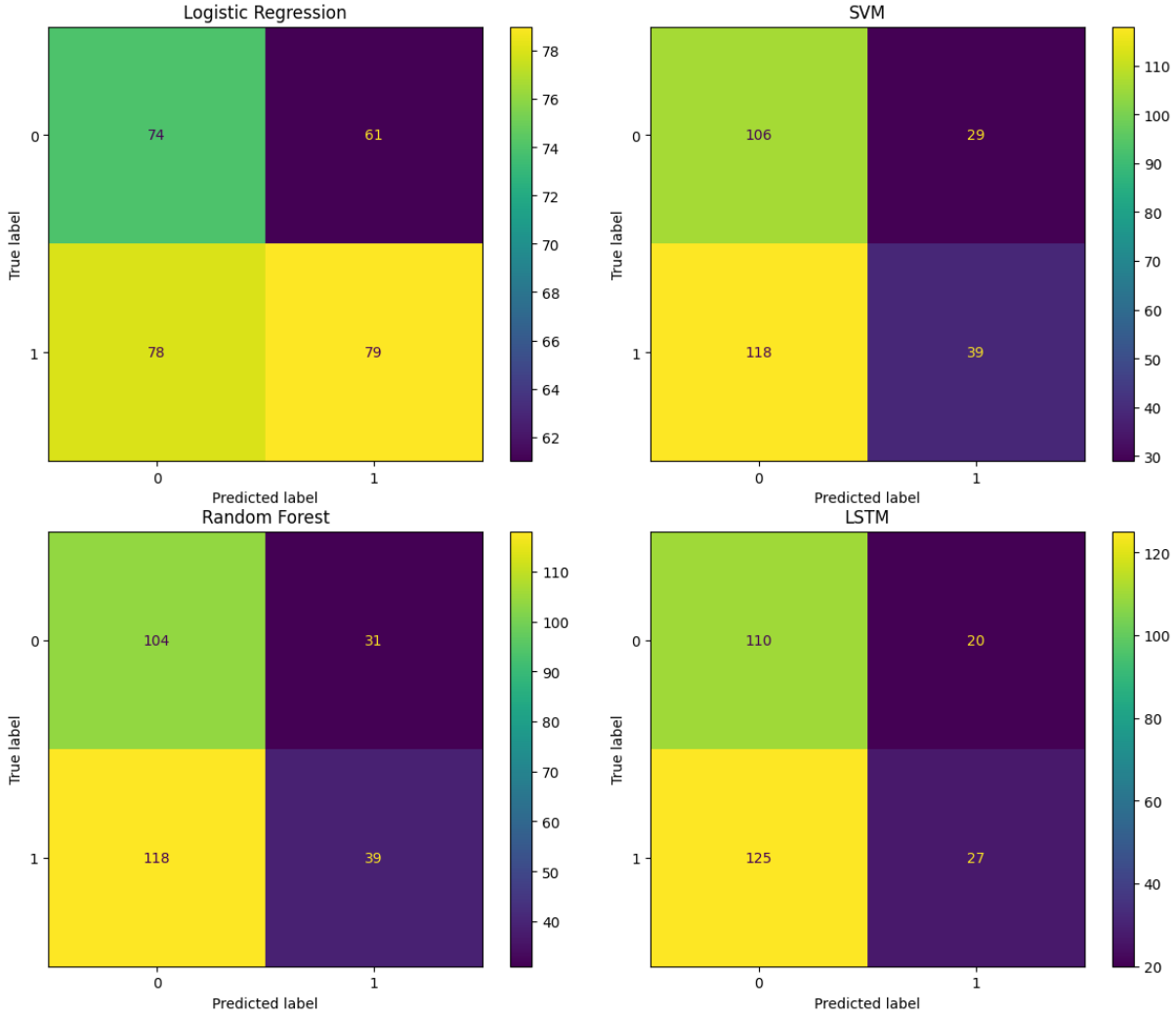


Figura 5.2: Confusion matrix dei modelli sul test set

Dall'analisi delle confusion matrix emergono comportamenti differenti tra i modelli.
La Regressione Logistica presenta una classificazione relativamente bilanciata: riconosce correttamente 74 giornate di ribasso e 79 giornate di rialzo, ottenendo un recall pari a 0.55 per la classe 0 e a 0.50 per la classe 1. Il modello commette inoltre 61 falsi positivi e 78 falsi negativi. Sebbene la sua accuracy rimanga inferiore alla baseline, la Regressione Logistica non si limita a prevedere sistematicamente la classe maggioritaria.
La Support Vector Machine mostra invece una maggiore propensione a riconoscere le giornate di ribasso. Il modello identifica correttamente 106 osservazioni della classe 0, raggiungendo per questa classe un recall pari a 0.79, ma riconosce soltanto 39 delle 157 giornate di rialzo. La matrice contiene quindi 29 falsi positivi e 118 falsi negativi.
La Random Forest presenta un comportamento simile alla SVM: classifica correttamente 104 giornate di ribasso e 39 giornate di rialzo, con un recall pari a 0.77 per la classe 0 e a 0.25 per la classe 1. Anche questo modello tende quindi a favorire la previsione dei ribassi, mostrando difficoltà nell'identificazione della classe positiva.
La rete LSTM accentua ulteriormente questa tendenza, riconoscendo correttamente 110 giornate di ribasso, ma soltanto 27 giornate di rialzo.
Nel complesso, le confusion matrix mostrano che nessun modello riesce a distinguere in modo pienamente efficace le due classi. La Regressione Logistica produce le predizioni più bilanciate, mentre SVM, Random Forest e LSTM risultano maggiormente orientate verso la classe 0.

5.4 Analisi grafica delle predizioni

Oltre alle metriche numeriche, sono state effettuate diverse analisi grafiche per confrontare le predizioni dei modelli con l'andamento reale del mercato.

5.4.1 Prezzo reale e segnali del modello

Sono stati rappresentati graficamente i prezzi reali del titolo Apple insieme ai segnali d'acquisto generati dal modello Random Forest, ovvero i giorni in cui il modello prevede un rialzo del prezzo.

5.4.2 Confronto tra valori reali e predetti

È stato effettuato un confronto diretto tra i valori reali della variabile target e le predizioni generate dalla Random Forest sulle prime 150 osservazioni del test.

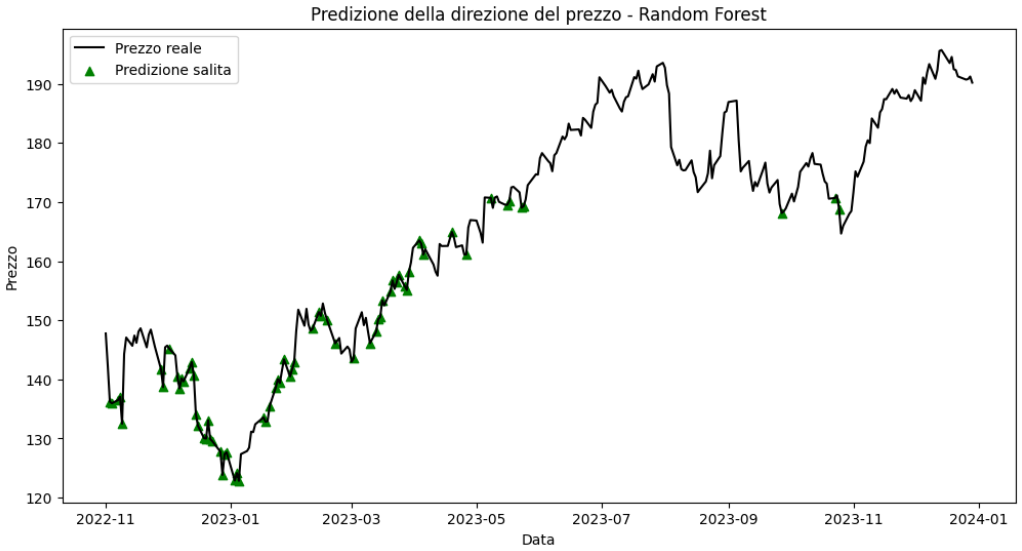


Figura 5.3: Andamento del prezzo di Apple e segnali di rialzo predetti dalla Random Forest

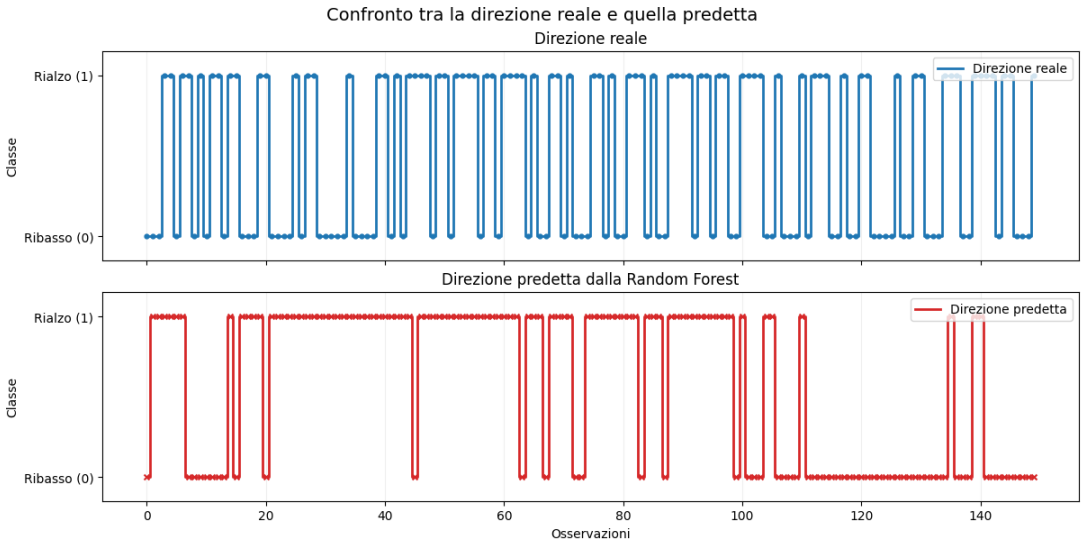


Figura 5.4: Confronto tra la direzione reale e quella predetta dalla Random Forest

5.5 Importanza delle feature

Per analizzare il contributo delle diverse variabili al processo decisionale del modello è stata studiata l'importanza delle feature calcolata dalla Random Forest. Questo indicatore misura quanto ciascuna variabile contribuisce mediamente alla riduzione dell'impurità nei nodi degli alberi decisionali [6].

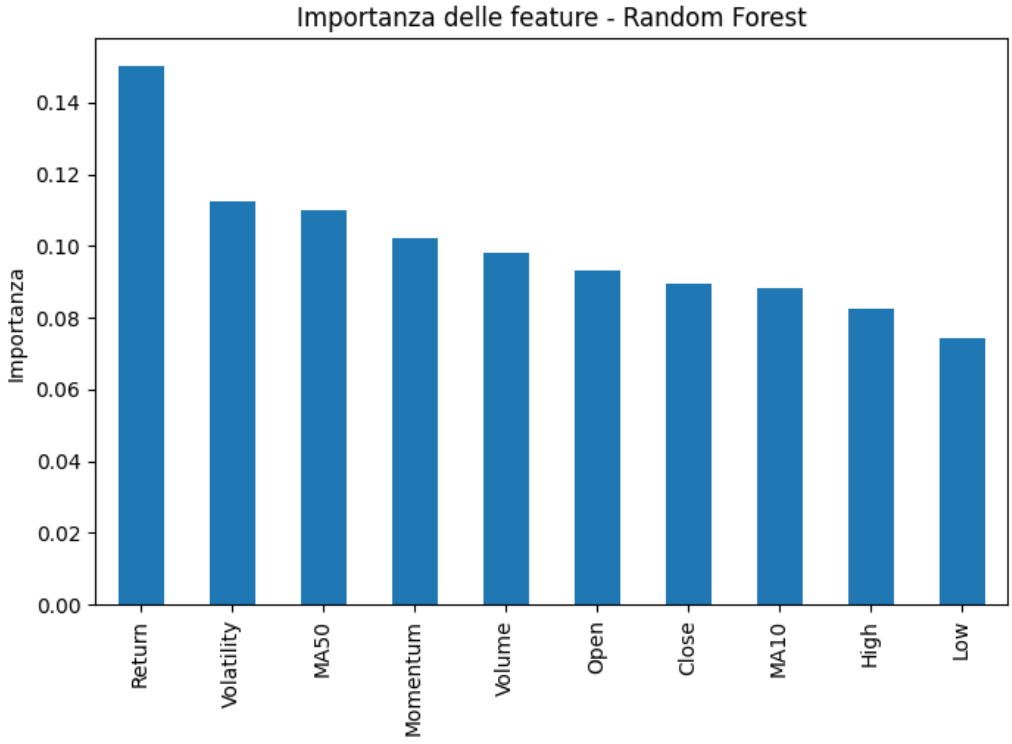


Figura 5.5: Importanza delle feature nella Random Forest

Dall'analisi dell'importanza delle feature emerge che il rendimento giornaliero (Return) rappresenta la variabile maggiormente utilizzata dalla Random Forest, con un'importanza pari a circa 0.15. Seguono la volatilità, la media mobile a 50 giorni e il momentum, con valori compresi approssimativamente tra 0.10 e 0.11. Le variabili originali, come Volume, Open, Close, High e Low, presentano invece un'importanza individuale più contenuta. Considerate complessivamente, le feature derivate contribuiscono per circa il 56.3% dell'importanza totale, mentre le variabili originali rappresentano il restante 43.7%. La Figura 5.5 suggerisce pertanto che gli indicatori ottenuti mediante feature engineering abbiano influenzato in misura rilevante le decisioni della Random Forest. Tuttavia, come mostrato dal confronto delle accuracy, questa maggiore importanza non si è tradotta in un miglioramento delle prestazioni sul test set, poiché il modello ha ottenuto risultati migliori utilizzando solamente le variabili originali. L'importanza attribuita a una feature descrive infatti quanto essa venga utilizzata nelle decisioni degli alberi, ma non garantisce necessariamente un incremento della capacità di generalizzazione del modello.

5.6 Analisi dell'impatto del feature engineering

Per valutare il contributo degli indicatori tecnici costruiti durante la fase di feature engineering, i modelli sono stati addestrati e valutati utilizzando due differenti configurazioni. Nella prima sono state impiegate esclusivamente le variabili originali del dataset, ossia Open, High, Low, Close e Volume. Nella seconda configurazione sono stati aggiunti il rendimento giornaliero, le medie mobili a 10 e 50 giorni, la volatilità e il momentum. In entrambi i casi è stata mantenuta la stessa suddivisione temporale tra training set e test set. Per la Regressione Logistica e la Support Vector Machine è stata inoltre applicata la standardizzazione delle feature mediante StandardScaler, addestrato esclusivamente sui dati di training.

Come mostrato nella Figura 5.6, l'effetto del feature engineering varia a seconda del modello considerato. La Regressione Logistica ha ottenuto un'accuracy del 49.3% utilizzando esclusivamente le variabili originali e del 52.4% con l'aggiunta degli indicatori tecnici, registrando un incremento di circa 3.1 punti percentuali. Anche la Support Vector Machine ha mostrato un leggero miglioramento, passando dal 48.3% senza feature engineering al 49.7% con l'impiego delle feature derivate.

Un comportamento differente è stato osservato per la Random Forest, la cui accuracy è diminuita dal 51.7% al 49.0% in seguito all'introduzione degli indicatori tecnici. In questo caso, le feature aggiuntive non hanno migliorato la capacità predittiva del modello e potrebbero aver introdotto informazioni ridondanti o poco utili rispetto alle variabili originali.

Nel complesso, il feature engineering ha prodotto un miglioramento dell'accuracy per la Regressione Logistica e, in misura più contenuta, per la SVM, mentre ha determinato un peggioramento per la Random Forest. Nessuna delle configurazioni analizzate è tuttavia riuscita a superare la baseline del 53.8%. I risultati indicano quindi che il contributo degli indicatori tecnici dipende dal modello utilizzato e rimane insufficiente per prevedere in modo affidabile la direzione giornaliera del prezzo azionario.

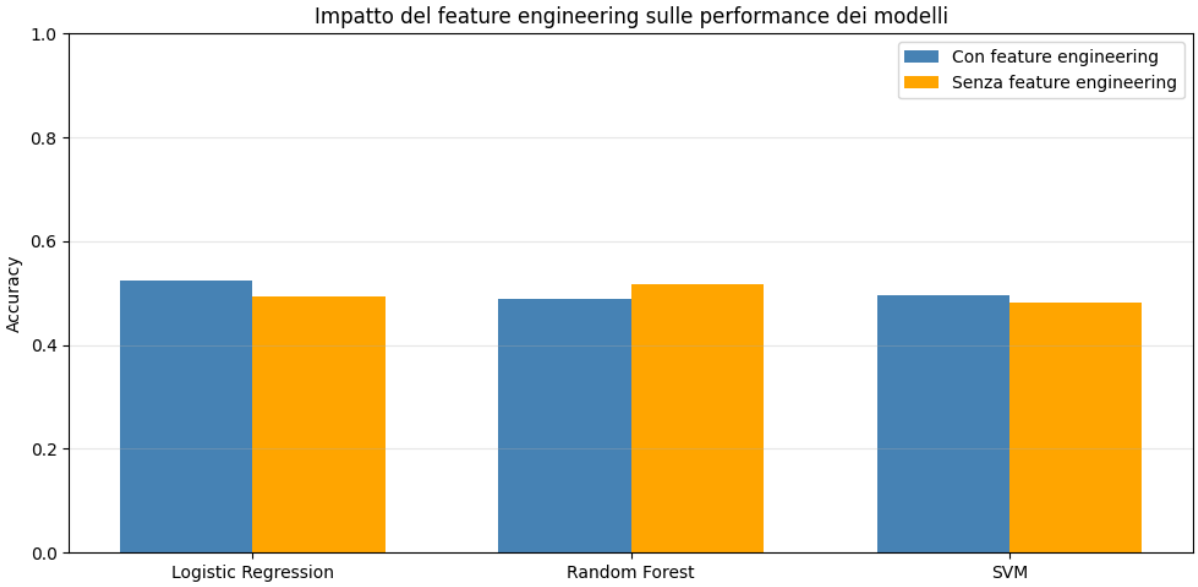


Figura 5.6: Confronto dell'accuracy dei modelli con e senza feature engineering

Capitolo 6

Discussione dei risultati

6.1 Interpretazioni e confronto delle performance

L'analisi dei risultati sperimentali evidenzia prestazioni complessivamente contenute per tutti i modelli considerati. Le accuracy ottenute sul test set variano dal 48.6% della rete LSTM al 53.8% della baseline, mostrando la difficoltà di prevedere la direzione giornaliera del prezzo azionario utilizzando esclusivamente dati storici di mercato. Questi risultati sono coerenti con la complessità e l'elevata rumorosità che caratterizzano le serie finanziarie.

La baseline ha registrato l'accuracy più elevata, pari al 53.8%, prevedendo sistematicamente la classe 1, corrispondente a un rialzo del prezzo. Tale risultato dipende dalla maggiore presenza di giornate di rialzo nel test set e non rappresenta una reale capacità di distinguere tra le due classi. La baseline, infatti, presenta un F1-score ponderato pari a 0.38 e non identifica alcuna giornata di ribasso.

La Regressione Logistica ha ottenuto un'accuracy del 52.4%, leggermente inferiore alla baseline, ma ha mostrato un comportamento più equilibrato. Il modello ha raggiunto valori ponderati di precision, recall e F1-score rispettivamente pari a 0.53, 0.52 e 0.52, classificando correttamente sia una parte delle giornate di rialzo sia una parte di quelle di ribasso. La standardizzazione delle feature ha quindi evitato che il modello si limitasse a prevedere la classe maggioritaria, pur senza consentirgli di superare la baseline in termini di accuracy.

La Support Vector Machine e la Random Forest hanno ottenuto accuracy rispettivamente pari al 49.7% e al 49.0%, con F1-score ponderati pari a 0.46 e 0.45. Entrambi i modelli hanno mostrato una maggiore propensione a riconoscere le giornate di ribasso, raggiungendo per la classe 0 valori di recall rispettivamente pari a 0.79 e 0.77. Questa capacità è stata però accompagnata da una maggiore difficoltà nell'identificazione delle giornate di rialzo, per le quali entrambi i modelli hanno ottenuto un recall pari a 0.25.

La rete LSTM ha registrato l'accuracy più bassa, pari al 48.6%, e un F1-score pon-

derato pari a 0.42. Anche questo modello ha privilegiato la previsione della classe 0, riconoscendo correttamente una parte consistente delle giornate di ribasso, ma mostrando difficoltà nell'identificazione dei rialzi. Nonostante la capacità teorica delle reti LSTM di apprendere dipendenze temporali, l'architettura utilizzata non ha prodotto un vantaggio rispetto ai modelli tradizionali nel contesto analizzato.

Nel complesso, nessun modello è riuscito a superare la baseline in termini di accuracy. Tuttavia, il confronto delle altre metriche e delle confusion matrix mostra che la Regressione Logistica ha fornito la classificazione più equilibrata, mentre SVM, Random Forest e LSTM si sono orientate maggiormente verso la previsione dei ribassi. I risultati confermano quindi che l'accuracy, considerata isolatamente, non è sufficiente per valutare adeguatamente modelli applicati a classi distribuite in modo non perfettamente uniforme [8].

6.2 Vantaggi e limiti dei modelli utilizzati

Ogni modello analizzato presenta vantaggi e limitazioni specifiche che è importante riconoscere esplicitamente.

La Regressione Logistica offre semplicità interpretativa, velocità di addestramento e facilità di implementazione. Tuttavia, la sua struttura lineare limita fortemente la capacità di rappresentare pattern complessi nei mercati finanziari, come dimostrato dai risultati nel presente lavoro.

La Support Vector Machine risulta efficace nei problemi di classificazione ad alta dimensionalità e permette di gestire relazioni non lineari tramite il kernel trick. Nel presente lavoro ha mostrato un comportamento più equilibrato rispetto agli altri modelli, riuscendo a identificare una parte delle giornate di ribasso.

La Random Forest presenta una buona capacità di generalizzazione e fornisce informazioni utili sull'importanza delle feature. Risulta inoltre meno sensibile al rumore rispetto ai singoli alberi decisionali. Tuttavia, come mostrato dai risultati, anche questo modello fatica a superare la soglia del 50% nell'accuracy su dati finanziari giornalieri.

Le reti LSTM rappresentano uno degli approcci più avanzati per l'analisi delle serie temporali. Tuttavia, i risultati ottenuti nel presente lavoro suggeriscono che la loro efficacia dipende fortemente dalla disponibilità di dataset ampi e da una fase di ottimizzazione accurata degli iperparametri [8].

6.3 Limiti della previsione nei mercati finanziari

Nonostante i risultati ottenuti, la previsione dei prezzi azionari rimane un problema particolarmente complesso. Come teorizzato dall'ipotesi dei mercati efficienti [7], i prezzi

di mercato incorporano in ogni momento tutte le informazioni disponibili, rendendo teoricamente impossibile ottenere un vantaggio sistematico basato sull'analisi dei soli dati storici.

I risultati ottenuti nel presente lavoro sono coerenti con questa ipotesi: tutti i modelli si avvicinano al 50% di accuracy, suggerendo che i pattern presenti negli indicatori tecnici utilizzati non contengano informazioni sufficienti per prevedere in modo affidabile la direzione futura dei prezzi.

Ulteriori limiti metodologici del presente lavoro riguardano:

- La scelta delle feature: le variabili di prezzo originali (Open, High, Low, Close) sono non stazionarie e i loro valori assoluti non sono direttamente informativi per la previsione della direzione futura. Un approccio più rigoroso utilizzerebbe esclusivamente variabili stazionarie come i rendimenti;

- L'ottimizzazione parziale degli iperparametri: per la Random Forest e la Support Vector Machine è stata effettuata una ricerca degli iperparametri mediante GridSearchCV e TimeSeriesSplit. La ricerca è stata tuttavia limitata alle configurazioni incluse nelle griglie definite e non è stata estesa alla Regressione Logistica e alla rete LSTM; ciò potrebbe aver limitato le prestazioni ottenibili;

- La dimensione del dataset: il periodo analizzato copre sei anni di dati giornalieri, una quantità sufficiente per i modelli classici ma potenzialmente limitante per la rete LSTM, che trae maggior beneficio da dataset più ampi;

- L'assenza di fattori esterni: il modello considera esclusivamente dati storici di prezzo e volume, escludendo informazioni potenzialmente rilevanti come dati macroeconomici, sentiment di mercato e notizie finanziarie.

Capitolo 7

Conclusioni

7.1 Sintesi del lavoro

Nel presente lavoro sono state analizzate e confrontate diverse tecniche di Machine Learning e Deep Learning applicate alla previsione della direzione dei prezzi azionari del titolo Apple Inc. (AAPL) nel periodo gennaio 2018 - dicembre 2023.

Dopo una fase iniziale di raccolta e preparazione dei dati storici, sono state sviluppate diverse feature derivate dall'analisi tecnica, tra cui rendimenti giornalieri, medie mobili, volatilità e momentum. Tali informazioni sono state utilizzate per addestrare e confrontare quattro modelli di classificazione supervisionata:

- Regressione logistica
- Support Vector Machine
- Random Forest
- Rete Neurale Ricorrente di tipo LSTM

Le prestazioni dei modelli sono state valutate attraverso diverse metriche di classificazione, tra cui accuracy, precision, recall e F1-score, integrate dall'analisi delle confusion matrix e dalla rappresentazione grafica delle predizioni.

I risultati ottenuti hanno evidenziato valori di accuracy compresi tra il 48.6% e il 53.8%, confermando la complessità del problema analizzato. La baseline ha registrato l'accuracy più elevata, pari al 53.8%, ma tale risultato è dovuto alla previsione sistematica della classe maggioritaria e non riflette una reale capacità di distinguere tra rialzi e ribassi. La Regressione Logistica ha ottenuto un'accuracy del 52.4%, mostrando tuttavia la classificazione più equilibrata delle due classi e il valore di F1-score più elevato. La Support Vector Machine e la Random Forest hanno mostrato una maggiore capacità di riconoscere le giornate di ribasso, accompagnata però da una minore efficacia nell'identificazione dei rialzi. La rete LSTM non ha invece evidenziato vantaggi significativi rispetto ai modelli tradizionali nel contesto specifico analizzato.

Nel complesso, nessuno dei modelli è riuscito a superare la baseline in termini di accuracy. Il lavoro ha quindi confermato l'elevata difficoltà della previsione dei mercati finanziari, caratterizzati da forte volatilità, rumore e continua variabilità temporale, fattori che limitano la capacità dei modelli di apprendere pattern stabili e generalizzabili utilizzando esclusivamente dati storici di mercato [7].

7.2 Sviluppi futuri

Il lavoro svolto può essere ulteriormente approfondito attraverso diverse possibili estensioni.

Un primo sviluppo potrebbe riguardare l'utilizzo di dataset più ampi e ricchi di informazioni aggiuntive, come indicatori macroeconomici, dati fondamentali delle aziende o informazioni provenienti da notizie finanziarie e social media. L'integrazione di queste fonti potrebbe fornire ai modelli un insieme informativo più completo rispetto ai soli dati storici di prezzo.

Un ulteriore miglioramento potrebbe derivare dall'ottimizzazione sistematica degli iperparametri tramite tecniche di cross-validation, con l'obiettivo di sfruttare appieno il potenziale predittivo dei modelli utilizzati, in particolare della rete LSTM.

Sarebbe inoltre interessante applicare architetture di Deep Learning più avanzate, come i modelli Transformer, che hanno mostrato risultati promettenti nell'analisi delle serie temporali finanziarie grazie ai meccanismi di attenzione. Infine, ulteriori studi potrebbero analizzare il comportamento dei modelli su titoli appartenenti a settori diversi o su mercati finanziari internazionali, al fine di valutare la generalizzabilità degli approcci utilizzati.

Bibliografia

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.

[2] C. W. Hsu, C. C. Chang e C. J. Lin, "A Practical Guide to Support Vector Classification," Department of Computer Science, National Taiwan University, 2016. [Online]. Disponibile: `https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf`. [Accesso: 17 maggio 2026].

[3] C. D. Manning, P. Raghavan e H. Schütze, "Support vector machines: The linearly separable case," in *Introduction to Information Retrieval*. Cambridge: Cambridge University Press, 2008. [Online]. Disponibile: `https://nlp.stanford.edu/IR-book/html/htmledition/support-vector-machines-the-linearly-separable-case-1.html`. [Accesso: 17 maggio 2026].

[4] J. Murel e E. Kavlakoglu, "What Are Classification Models?," IBM, 31 luglio 2024. [Online]. Disponibile: `https://www.ibm.com/think/topics/classification-models`. [Accesso: 17 maggio 2026].

[5] T. S. Reddy, R. Vinayakumar e K. P. Soman, "Machine Learning Algorithms, Real-World Applications and Research Directions," *SN Computer Science*, vol. 2, n. 3, pp. 1–14, 2021.

[6] Scikit-learn Developers, "RandomForestClassifier," *Scikit-learn Documentation*. [Online]. Disponibile: `https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html`. [Accesso: 17 maggio 2026].

[7] E. F. Fama, "Efficient Capital Markets: A Review of Theory and Empirical Work," *Journal of Finance*, vol. 25, n. 2, pp. 383–417, 1970.

[8] T. Fischer e C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, n. 2, pp. 654–669, 2018.

[9] J. Patel, S. Shah, P. Thakkar e K. Kotecha, "Predicting stock and stock price index movement using machine learning techniques," *Expert Systems with Applications*, vol. 42, n. 1, pp. 259–268, 2015.

[10] R. V. Arjun, "yfinance: Yahoo! Finance market data downloader," *PyPI*, 2019. [Online]. Disponibile: `https://pypi.org/project/yfinance/`. [Accesso: 17 maggio 2026].

[11] J. J. Murphy, *Technical Analysis of the Financial Markets*. New York: New York Institute of Finance, 1999.

[12] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Disponibile: `https://www.tensorflow.org/`. [Accesso: 17 maggio 2026].